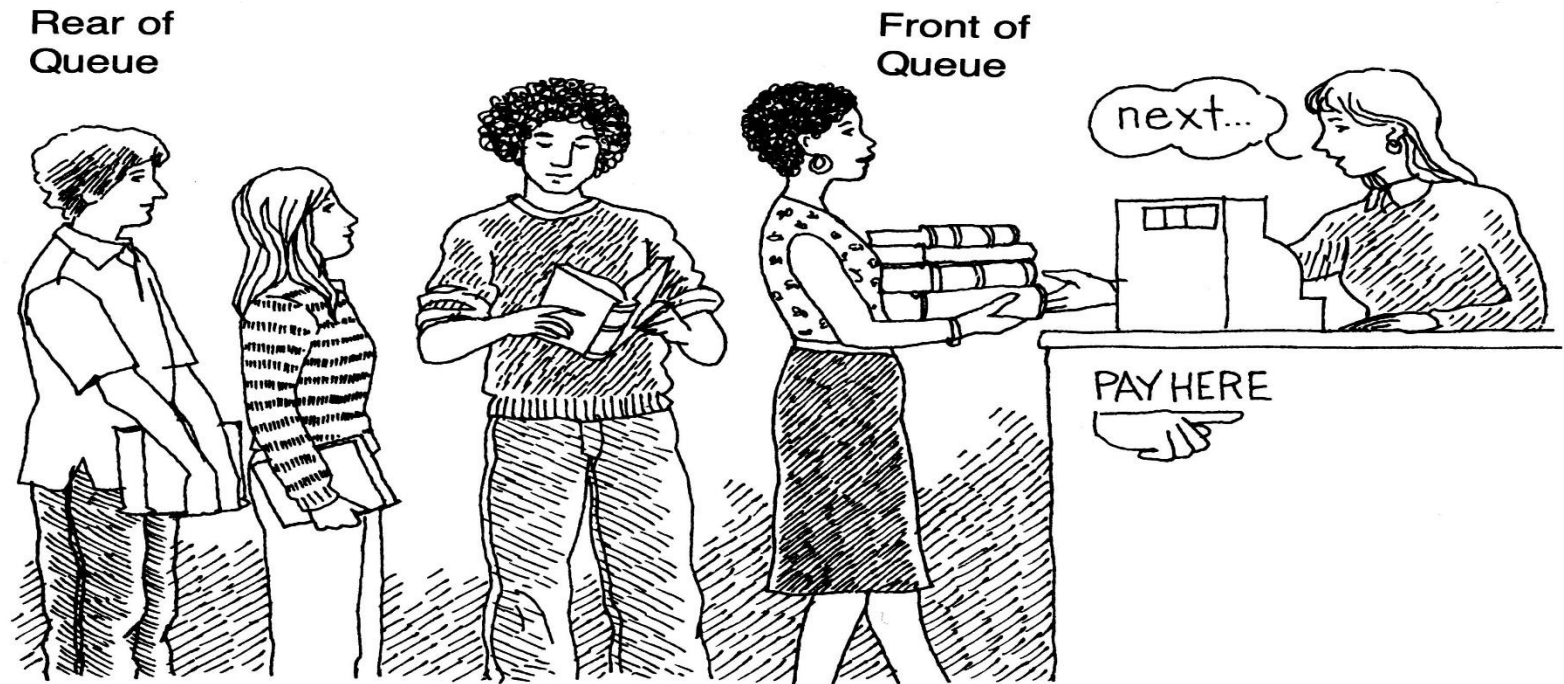


Outline

- Queue
- Implementations of the Queue
 - Pointer-Based Implementation
 - Array-Based Implementation
- Applications
 - Reading a String of Characters
 - Recognizing Palindromes

What is a queue?

- It is an ordered group of homogeneous items or elements.
- Queues have two ends:
 - Elements are added at one end.
 - Elements are removed from the other end.
- The element added first is also removed first (FIFO: First In, First Out).



ADT Queue Operations

- Create an empty queue
- Destroy a queue
- Determine whether a queue is empty
- Add a new item to the queue
- Remove from the queue the item that was added earliest
- Retrieve from the queue the item that was added earliest

Overflow and Underflow

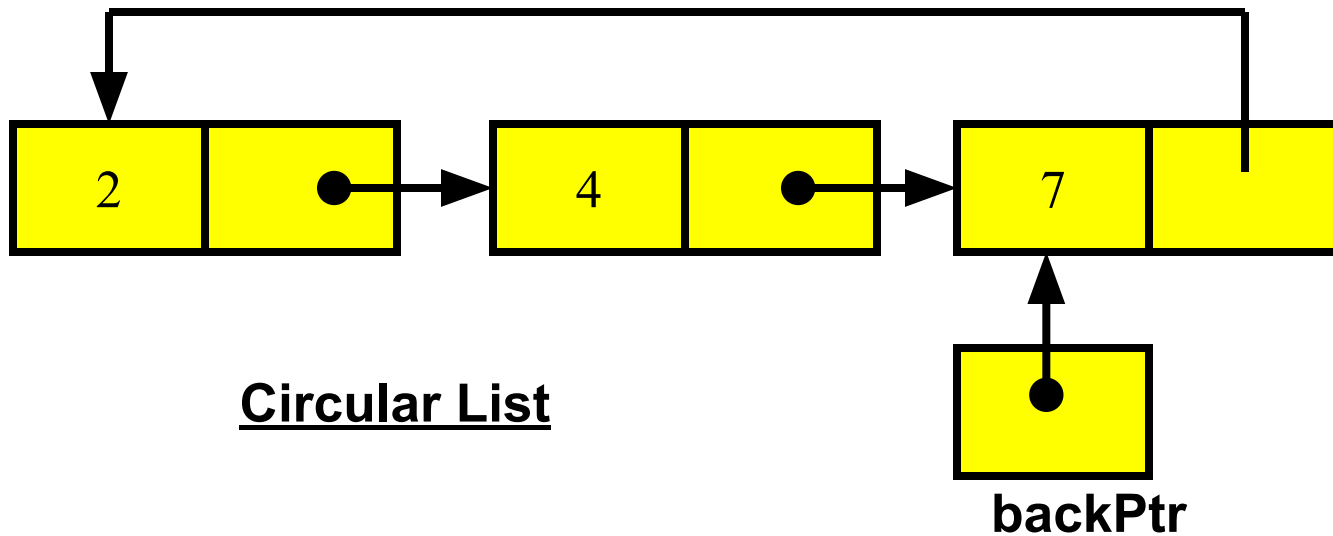
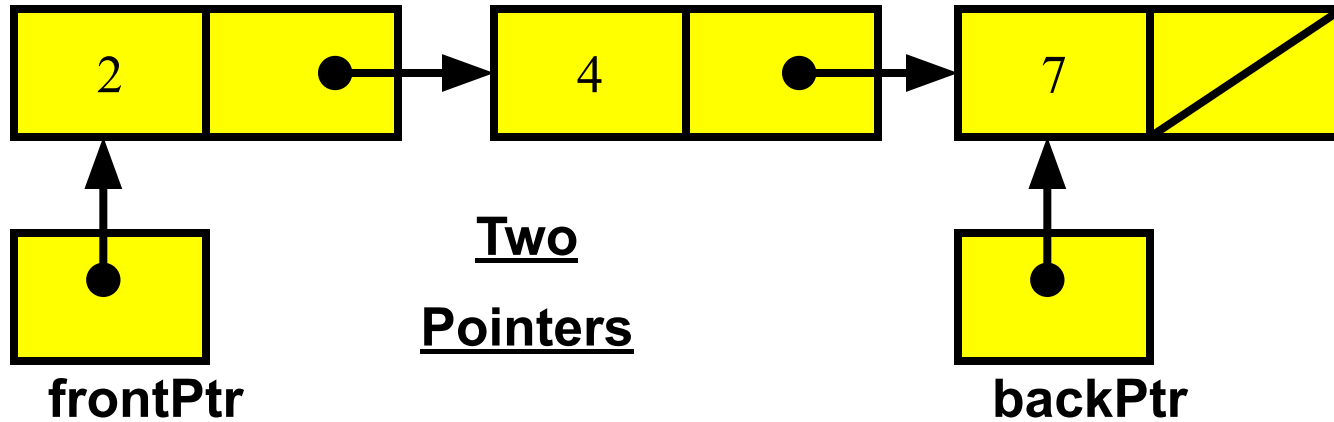
- The condition resulting from trying to add an element onto a full queue.(overflow)
- The condition resulting from trying to remove an element from an empty queue. (underflow)

Implementations of the Queue

- Array Based Implementation
- Pointer Based Implementation

The Choice Depends on Whether Your Problem can use Fixed-Size Queue That an Array-Based Implementation Provides.

Pointer Based Implementation



Pointer Based Implementation

```
typedef desired-type-of-queue-item QueueItemType;
```

```
class Queue
```

```
{
```

```
public:
```

```
// constructors and destructor:
```

```
    Queue( );                // default constructor
```

```
    Queue (const Queue& Q); // copy constructor
```

```
    ~Queue( );               // destructor
```

Pointer Based Implementation

// Queue operations:

```
bool isEmpty( ) const ;
```

```
void enqueue (QueueItemType newItem);
```

```
void dequeue( );
```

```
void dequeue(QueueItemType& queueFront);
```

```
void getFront(QueueItemType& queueFront) const;
```

Pointer Based Implementation

private:

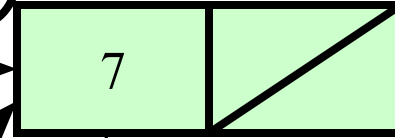
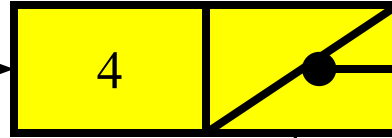
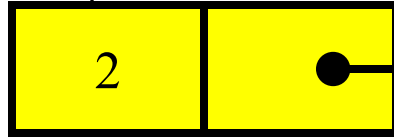
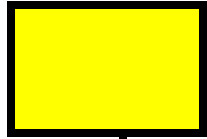
```
    struct QueueNode
    {
        QueueItemType item;
        QueueNode* next;
    }; // end struct
    QueueNode *frontPtr;
    QueueNode *backPtr;
}; //end class
// End of header file.
```

Pointer Based Implementation

```
Queue::Queue( ) : backPtr(NULL), frontPtr(NULL)
{
} // end default constructor
Queue::~~Queue( )
{
    while( !isEmpty( ) )
        dequeue( );
    // Assertion: frontPtr and backPtr equal NULL
} // end destructor
```

Inserting an item

frontPtr

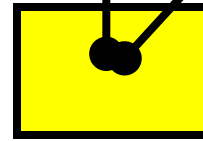


1

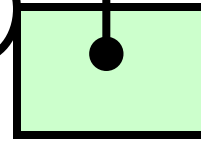
2

3

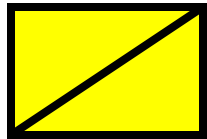
1. newPtr->next = NULL;
2. backPtr->next = newPtr;
3. backPtr = newPtr;



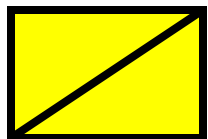
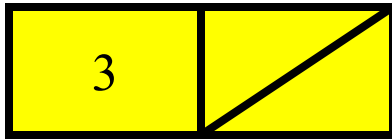
backPtr



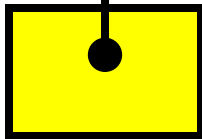
newPtr(points to new node)



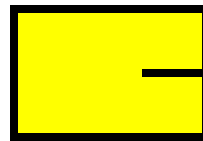
frontPtr



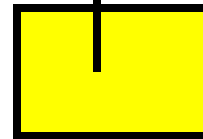
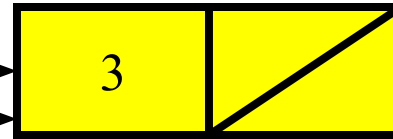
backPtr



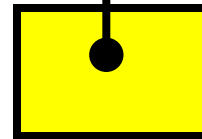
newPtr



frontPtr



backPtr



newPtr

frontPtr = newPtr;
backPtr = newPtr;

Pointer Based Implementation

```
bool Queue::isEmpty( ) const
{
    return bool (backPtr == NULL);
} // end isEmpty
```

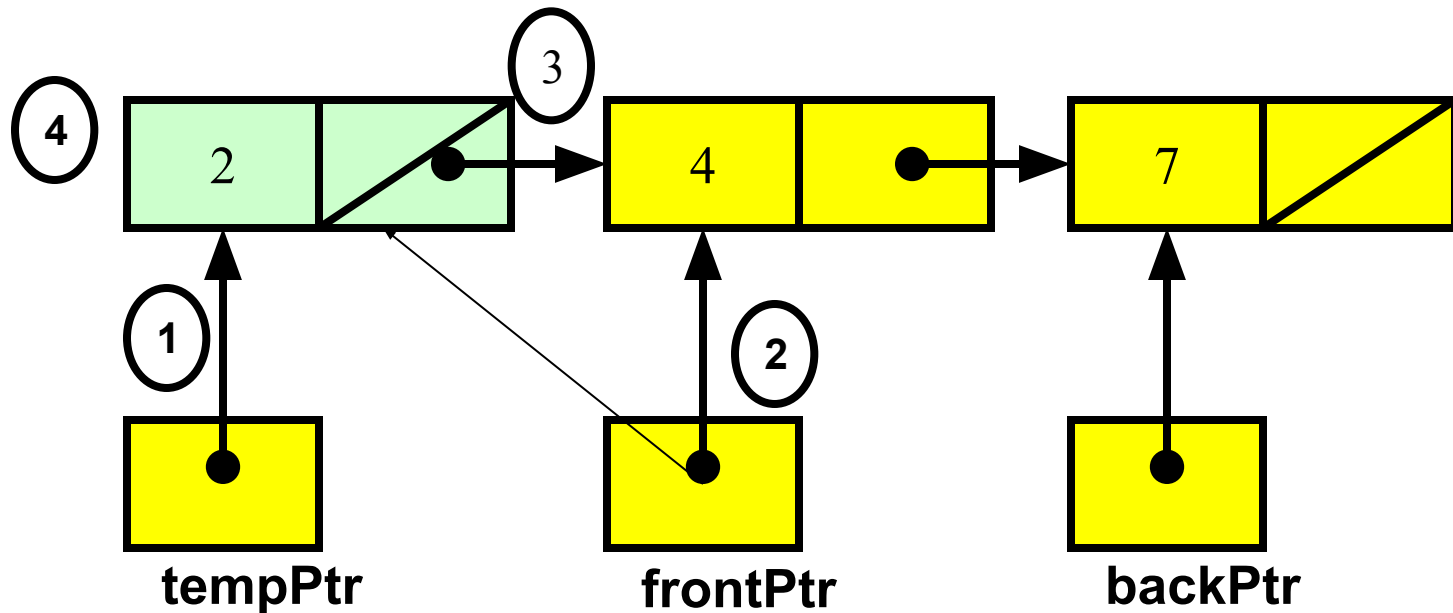
```
void Queue::enqueue(QueueItemType newItem)
{
    // create a new node
    QueueNode * newPtr = new QueueNode;

    if (newPtr == NULL)    //check allocation
        cout<<endl<<"cannot add item";
    else
    { // allocation successful; set data portion of new node
        newPtr->item = newItem;
```

Pointer Based Implementation

```
// insert the new node
if (isEmpty( ))
    // insertion into empty queue
    frontPtr = newPtr;
else
    // insertion into nonempty queue
    newPtr->next = backPtr;
backPtr = newPtr; // new node is at back
} // end if
} // end enqueue
```

Deleting an item from a queue of more than one item



1. `tempPtr = frontPtr;`
2. `frontPtr = frontPtr->next;`
3. `Tempptr->next = NULL;`
4. `delete tempPtr;`

Pointer Based Implementation

```
void Queue::dequeue( )
{
    if ( isEmpty( ))
        cout<<endl<<"cannot remove item";
    else
    { // queue is not empty; remove front
        QueueNode *tempPtr = frontPtr;
        if (frontPtr == backPtr) // special case?
        { // yes, one node in queue
            frontPtr = NULL;
            backPtr = NULL;
        }
        else
            frontPtr = frontPtr ->next;
        tempPtr->next = NULL; // defensive strategy
        delete tempPtr;
    } // end if
} // end dequeue
```

Pointer Based Implementation

```
void Queue::dequeue( QueueItemType& queueFront)
{
    if ( isEmpty( ))
        cout<<endl<<"cannot remove item";
    else
    { // queue is not empty; remove front
        queueFront = frontPtr->item;
        dequeue( );    //delete front
    } // end if
} // end dequeue
```

Pointer Based Implementation

```
void Queue::getFront(QueueItemType& queueFront) const
{
    if (isEmpty( ))
        cout<<endl<<"empty queue, cannot getFront";
    else
        // queue is not empty; retrieve front
        queueFront = frontPtr->item;
} // end getFront
// End of implementation file.
```

Array Based Implementation

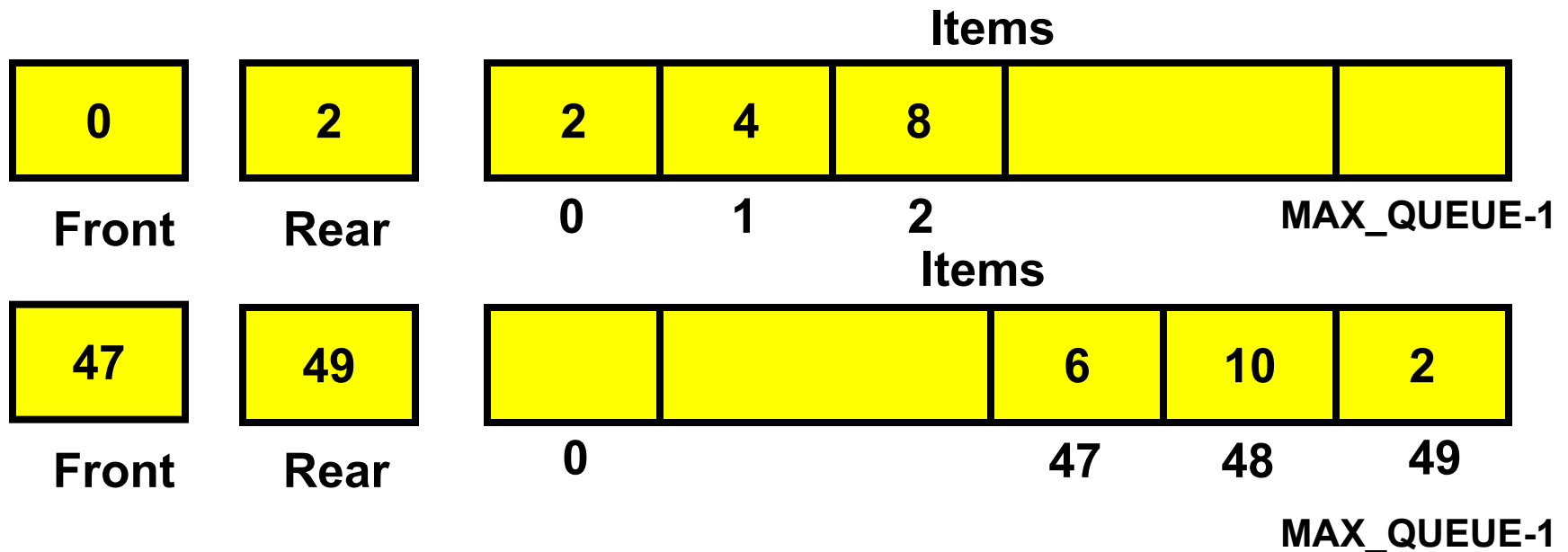
- `const int MAX_QUEUE = maximum-size-of-queue;`
- `typedef desired-type-of-queue-item queueItemType;`

- `queueItemType Items[MAX_QUEUE];`
- `intFront;`
- `intRear;`

- Initially Front = 0, Rear = -1
- Adding an Item : Increment Rear, Place Item in Items[Rear]
- Removing an Item : Increment Front
- Queue is Empty When Rear is Less Than Front.
- Queue is Full When Rear Equals MAX_QUEUE-1

Array Based Implementation

1. After a Sequence of Additions and Removals the Items in Queue will Drift Down the High End of The Array
2. Rear Could Equal $\text{MAX_QUEUE}-1$ Even When the Queue Contains Only a Few Items
3. This Rightward Drift Could be a Problem

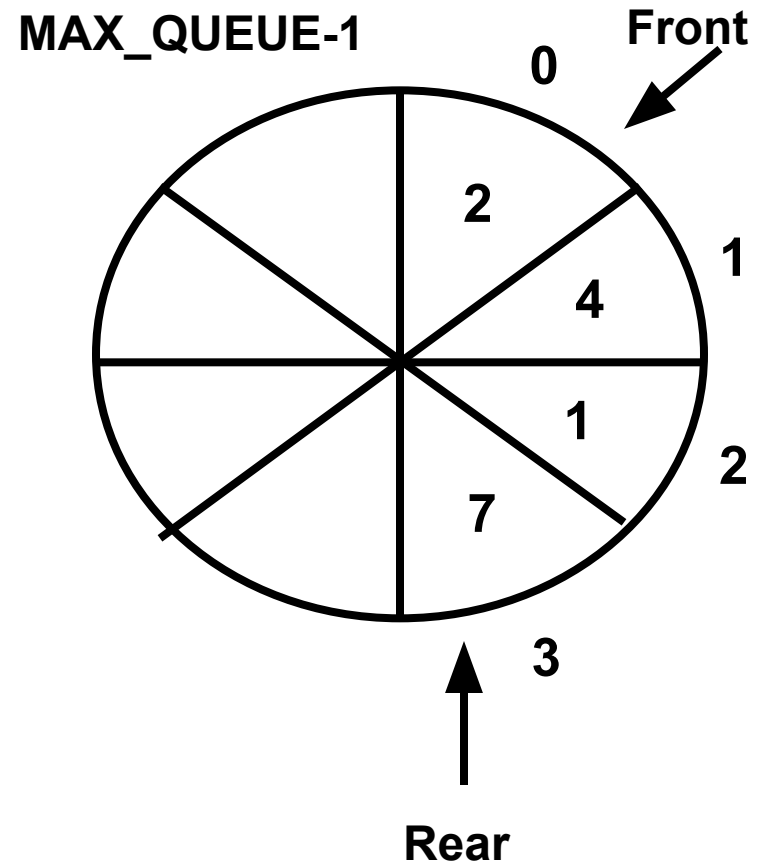


Array : Implementation issues

- Implement the queue as a *circular structure*.
- How do we know if a queue is full or empty?
- Initialization of *front* and *rear*.
- Testing for a *full* or *empty* queue.

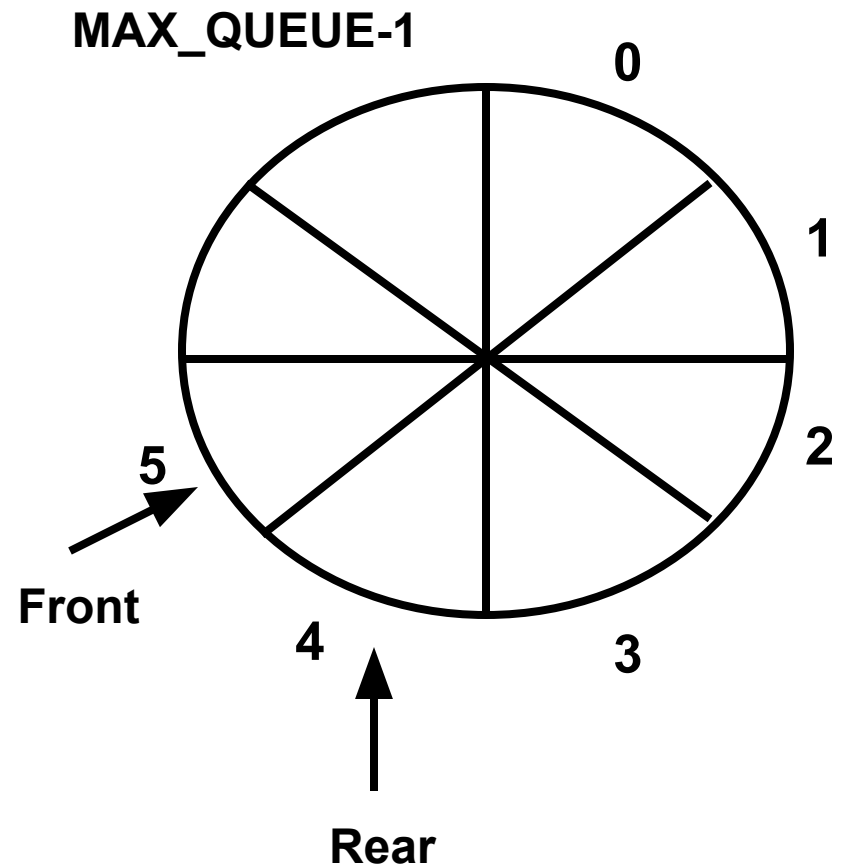
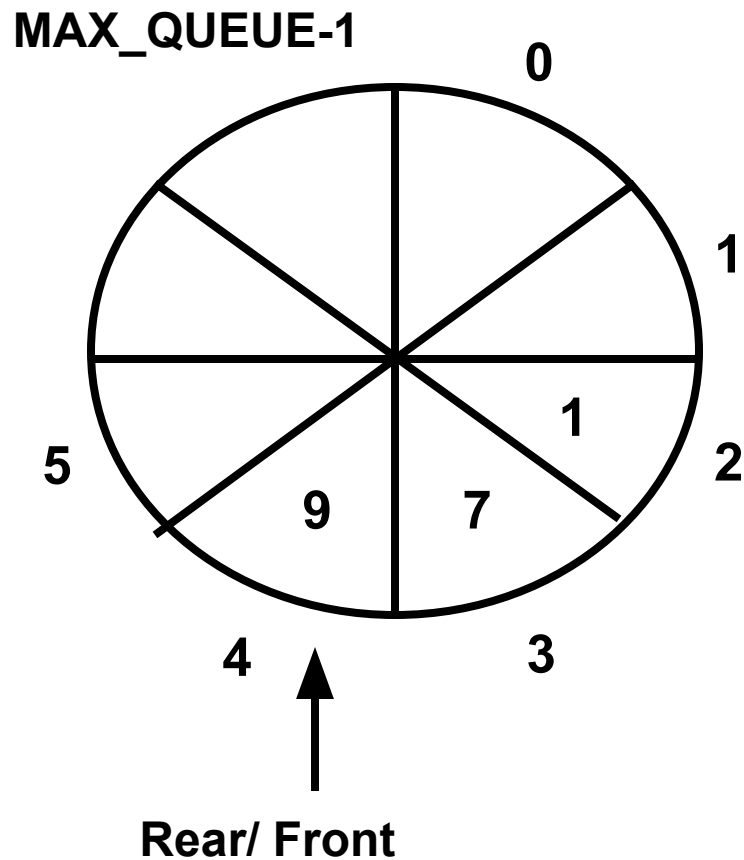
Array Based Implementation

- Array could be Viewed as Circular.
- When Either front or Rear Advances past Location MAX_QUEUE-1, it Wraps Around to Position Zero.
- Detection of Queue Empty and Full.
- Queue Empty if front is One Slot Ahead of Rear ??



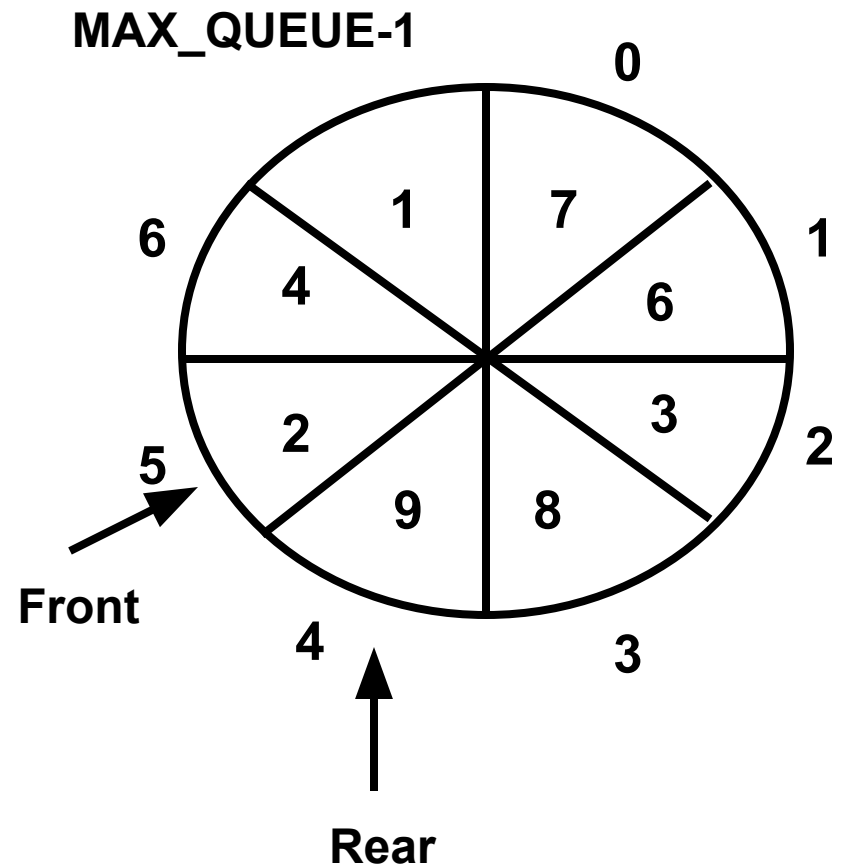
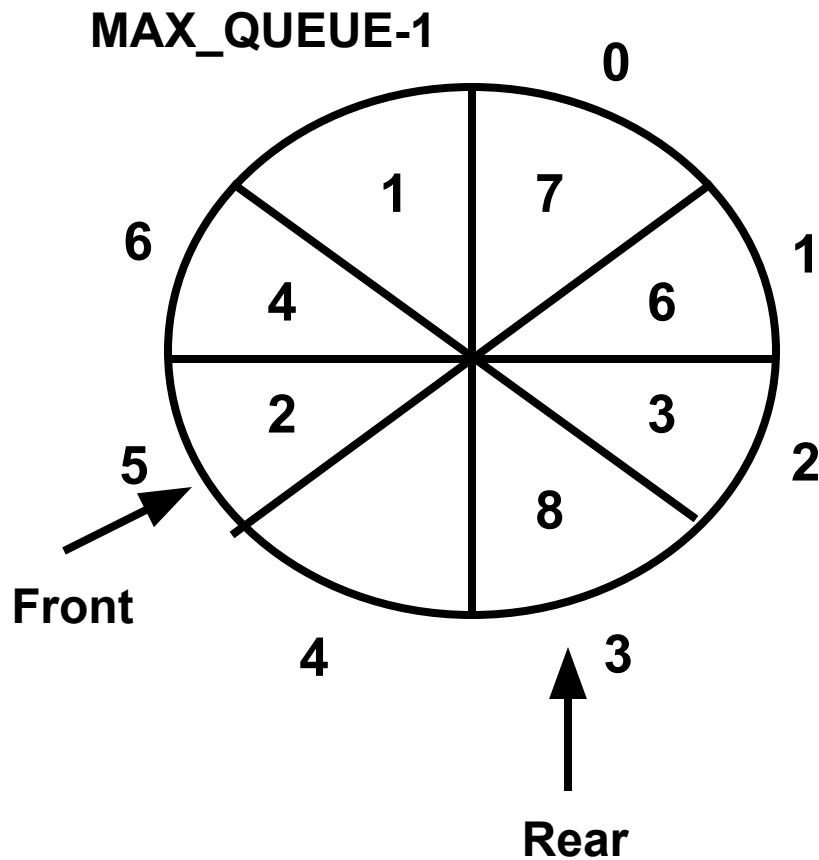
Array Based Implementation

Queue with single element : Remove element : queue becomes empty



Array Based Implementation

Queue with single empty slot : Add 9 : queue becomes full



Array Based Implementation

```
class Queue
{
public:
    Queue(); // default constructor
    Queue(const Queue& Q);
    ~Queue( );
    bool QueueIsEmpty();
    void QueueAdd(queueItemType NewItem, bool& Success);
    void QueueRemove(bool& Success);
    void GetQueueFront(queueItemType& QueueFront,
                       bool& Success);
private:
    queueItemType Items[MAX_QUEUE];
    int          Front;
    int          Rear;
    int          Count;
}; // end class
```

Array Based Implementation

```
Queue::Queue(): Front(0), Rear(MAX_QUEUE-1), Count(0)
{
} // end default constructor
bool Queue::QueueIsEmpty( )
{
    return (Count == 0);
} // end QueueIsEmpty
void Queue::QueueAdd(queueItemType NewItem,
                    bool& Success)
{
    Success = (Count < MAX_QUEUE);
    if (Success)
    { // queue is not full; insert item
        Rear = (Rear+1) % MAX_QUEUE;
        Items[Rear] = NewItem;
        ++Count;
    } // end if
} // end QueueAdd
```

Array Based Implementation

```
void Queue::QueueRemove(bool& Success)
```

```
{ Success = !QueueIsEmpty( );  
  if (Success)  
  { // queue is not empty; remove front  
    Front = (Front+1) % MAX_QUEUE;  
    --Count;  
  } // end if  
} // end QueueRemove
```

```
void Queue::GetQueueFront(queueItemType& QueueFront,  
                          bool& Success)
```

```
{  
  Success = !QueueIsEmpty( );  
  if (Success)  
    // queue is not empty; retrieve front  
    QueueFront = Items[Front];  
} // end GetQueueFront  
// End of implementation file
```

Variations of Array Based Implementation

- **IsFull Flag to Distinguish Between Empty and Full Conditions**
- **Use Locations One Less Than the Size of Array**
 - Initially $\text{Front} = \text{Rear} = 0$: Queue is Empty
 - If $\text{Front} = \text{Rear} + 1$: Queue is Full
 - Before Adding a Value We Check For Condition $\text{Rear} + 1 == \text{Front}$

Example: Reading a String of Characters

- When You Enter Characters at a Keyboard, the System Must Retain Them in the Order in Which you Typed Them.
- It Could Use a Queue for This Purpose.

Example: Reading a String of Characters

- //read a string of characters from a single line of input into a //queue
- Q.CreateQueue()
 while (not end of line)
 { Read a new character Ch
 Q.QueueAdd (Ch, Success)
 }
- Once the characters are in a queue, the system can process them as necessary

Example: Reading a String of Characters

- //convert digits in a queue Q into a
- //decimal integer N
- $N = 0$
- //ignore leading blanks, if any
 - do
 - {
 - Q.GetQueueFront(Ch, Success)
 - Q.QueueRemove (Success)
 - }while (Ch is blank)
 - //assertion: Ch contains first digit
 - //compute N from digits
 - do
 - $\{N = 10 * N + \text{integer that Ch represents}\}$
 - Q.GetqueueFront (Ch, Success)
 - Q.QueueRemove(Success)
 - }while (Success and Ch is a digit)
 - //N is result

Example: recognizing palindromes

- A *palindrome* is a string that reads the same forward and backward.

Able was I ere I saw Elba

- We will read the line of text into both a stack and a queue.
- Compare the contents of the stack and the queue character-by-character to see if they would produce the same string of characters.

Example: recognizing palindromes

a
b
l
E
⋮
e
l
b
A

Stack

A	b	l	e				E	l	b	a
---	---	---	---	--	-------	--	---	---	---	---

Queue

Example: recognizing palindromes

IsPal(Str)

//Determines whether the string Str is a palindrome.

//initialize a queue and a stack

Q.CreateQueue()

Q.CreateStack()

//insert each character of the string into both the queue //and the stack

for(I=0 through strlen(Str)-1)

{ NextChar = Str[I]

 Q.QueueAdd(NextChar, Success)

 S.Push(NextChar, Success)

} //end for

//compare the queue characters with the stack characters

CharacterAreEqual = TRUE

Example: recognizing palindromes

```
while( Q is not empty and CharactersAreEqual)
{  Q.GetQueueFront(QueueFront, Success )
   S. GetStackFront(StackTop, Success )

   if ( QueueFront equals StackTop)
   {
      Q.QueueRemove(Success)
      S.Pop(Success)
   }    //end if
   else
      CharacterAreEqual = FALSE
} //end while
return CharactersAreEqual
```

Priority Queue (Extra Slide for Information)

- Is a Value-Oriented ADT and is More Sophisticated Than Position-Oriented ADTs Like Stacks and Queues
- Priority Value Indicates, For Example a Task's Priority for Completion
- A Priority Queue can be Viewed as a Sequence of Values Which are Sorted According to Values
- The Rules to Determine Order:
 - An Element of Higher Priority is Processed Before any Element of Lower Priority
 - Two Elements With the Same Priority are Processed According to the Order in Which They Were Added to the Queue
- Representations
 - Single Queue
 - Multi-Level Queue

Priority Queue

- **Implementations**

- **Sorted Array/List**

- Difficult to Insert Value
 - Very Easy to Find Value (MaxPri)

- **Unsorted Array / List**

- Easy to Insert Value
 - Difficult to Find Value (MaxPri)

- **Array Versus List Implementation**